

学 号:	2025308025
------	------------

武汉理工大学

基于多智能体与检索增强生成的研究生 智能学习助手

学 院 计算机与人工智能学院

专 业 计算机技术

班 级 研 252

学 号 2025308025

姓 名 锡博然

2026 年 06 月 20 日

目录

摘要..... 3

一、选题背景与设计思想..... 3

 1.1 问题定义..... 3

 1.2 现有方案不足..... 3

 1.3 项目研究价值..... 4

 1.4 整体技术路线..... 4

二、Specs 规格文档..... 5

 2.1 产品规格说明书（Product Spec）..... 5

 2.2 架构规格说明书（Architecture Spec）..... 6

 2.3 API 接口规格说明书（API Spec）..... 7

三、系统架构与设计..... 8

 3.1 整体分层架构..... 8

 3.2 Agent 交互流程..... 9

 3.3 全链路数据流设计..... 10

四、关键实现与代码展示..... 10

 4.1 Agent 核心循环..... 10

 4.2 工具定义..... 12

 4.3 配置文件..... 13

 4.4 AI IDE 使用..... 13

五、测试与评估..... 14

 5.1 功能测试..... 14

 5.2 Agent 行为评估..... 14

 5.3 Demo 展示..... 15

六、系统升级与扩展..... 18

七、课程总结..... 19

摘要

本报告在研究生智能学习助手开发的基础上，提出了一种结合多智能体协作与检索增强生成（RAG）技术的新型框架，以提升学习问答的准确性和实用性。高等教育数字化转型已成为提升教育质量和教育公平的关键战略，国家智慧教育平台注册用户已突破 1.64 亿，教育数字化进入深度应用阶段。生成式大模型技术的快速发展使得多模态、多交互的教育智能体成为可能。单一智能体在需要多角色协同的复杂学习任务中仍存在视角单一、功能单一等局限，因此本研究探讨了一个由路由 Agent 协调多个专业 Agent（问答、学习规划、论文检索等）和知识检索与记忆模块构成的系统设计。

该系统对用户提问首先通过路由 Agent 识别意图，再结合知识检索为 LLM 提供专业背景信息，最后由相应子 Agent 生成回答。通过功能测试与案例验证，系统在问答准确率和工具调用正确率方面表现良好，回答符合知识库事实、未出现“幻觉”。本报告严格按照课程要求分为七大章节，涵盖选题背景、规格文档、架构设计、关键实现、测试评估、扩展规划与课程总结，完整呈现项目全周期成果。

一、选题背景与设计思想

1.1 问题定义

随着人工智能与教育信息化的推进，高等教育逐步向数字化、智能化转型，在线学习平台打破了传统教育的时空限制，但也催生了对智能化、专业化学习辅助工具的需求。当前研究生学习场景仍存在显著痛点：一是**教务信息碎片化**，课表、考试、课件、作业等学习信息分散在不同平台，学生信息获取成本高、效率低；二是**专业问答可信度低**，通用大模型依赖固有参数知识，面对细分专业知识点易产生幻觉，答案不可溯源、无法满足课程学习需求；三是**个性化服务缺失**，现有工具无长期用户记忆能力，无法结合学生学习情况提供定制化服务，交互效率低下。

大模型技术为教育智能辅助提供了技术支撑，但单智能体角色单一、能力有限，难以处理复杂的综合性学习任务。而多智能体系统通过多主体分工协作，能够高效解决复杂场景问题，为解决上述学习痛点提供了全新技术路径。

1.2 现有方案不足

目前主流学习辅助方案均存在明显短板，无法适配研究生综合学习需求：

- 传统教务系统**：交互形式固化，仅支持结构化固定查询，无法理解自然语言提问；各业务数据相互孤立，无法实现一站式学习服务。

2. **通用大模型**：专业知识准确率低、幻觉问题突出，回答缺乏依据与可溯源性；无法对接实时教务数据，信息时效性差。

3. **单智能体系统**：单主体承载全部学习任务，工具繁杂易出现调用错误，模型负载过高，复杂任务推理稳定性差，不符合工程化分工设计原则。

综上，现有方案无法同时满足实时教务查询、可信专业答疑、个性化服务、稳定处理多任务的核心需求，亟需一套定制化、解耦可扩展的教育多智能体解决方案。

1.3 项目研究价值

本项目探索了多智能体分工协作与检索增强生成技术在高等教育场景的融合应用方案，提出了“路由分发+专属 Agent+工具调用+RAG 增强”的四层智能体架构，验证了专业化分工对提升 Agent 任务准确率的作用，为教育类智能体系统的设计提供了可复用的架构参考。相关研究也印证了多智能体系统在教育场景中具备单 Agent 无法替代的协作优势。

系统面向研究生真实学习场景，整合课程查询、考试提醒、作业跟踪、知识答疑、学习规划五大核心能力，一站式覆盖研究生日常学习需求；通过工具调用与 RAG 双重机制保障答案准确性，显著降低幻觉风险；双轨记忆系统实现跨会话的个性化服务，有效提升学习信息获取效率与学习体验。

1.4 整体技术路线

本项目采用分层解耦的技术路线，从底层到上层依次为模型层、工具层、编排层、数据层与表现层，各层通过标准化接口交互，保障系统的可维护性与可扩展性：

1. **模型层**：选用通义千问 qwen-turbo 作为核心推理引擎，通过 OpenAI 兼容接口接入，提供自然语言理解、工具调用决策、答案生成能力。
2. **工具层**：封装课程表查询、考试信息查询、作业状态查询、学习建议、知识库检索五类标准化工具函数，作为系统与外部数据的交互入口。
3. **编排层**：基于 LangGraph 框架实现单 Agent 工具调用循环与多 Agent 路由分发，通过状态机管理任务流转，实现推理-执行-生成的完整闭环。
4. **数据层**：采用 ChromaDB 向量数据库存储课程知识库，支持语义相似度检索；采用 JSON 文件持久化用户画像，实现长期记忆存储。
5. **表现层**：基于 Streamlit 框架构建 Web 交互界面，提供零门槛的对话式交互体验。

二、Specs 规格文档

2.1 产品规格说明书 (Product Spec)

2.1.1 产品定位与目标用户

本产品是一款面向在校研究生的智能学习助手，以自然语言对话为核心交互方式，整合教务信息查询、专业知识答疑、个性化学习规划三大能力，致力于成为研究生的一站式学习辅助工具。核心目标用户为全日制研究生，次要用户包括课程助教与教学管理人员。

2.1.2 功能需求规格

功能模块	功能描述	优先级
课程信息查询	支持自然语言查询本周课程表、指定课程考试安排、作业提交状态、课程学习建议	P0
知识库智能问答	基于课程文档知识库，解答专业概念、算法原理、架构设计等学术问题，答案可溯源	P0
个性化学习规划	根据课程安排、考试时间、作业进度，自动生成定制化复习计划与学习路径建议	P1
双轨记忆系统	短期记忆保留对话上下文，长期画像记录用户专业、修读课程、考试安排等结构化信息	P1
多智能体分发	自动识别问题类型，分发至课程助手、论文助手、规划助手三类专属 Agent 处理	P1
系统指令支持	支持查看画像、重置对话、重置画像、手动检索等特殊指令操作	P2

2.2 架构规格说明书（Architecture Spec）

2.2.1 设计原则

系统遵循五大核心设计原则：一是**单一职责**，各模块、Agent 任务单一、边界清晰，降低使用与维护成本；二是**分层解耦**，各层级通过标准接口交互，层内迭代不影响整体架构；三是**可扩展**，预留工具、Agent、知识库扩展接口，支持业务平滑升级；四是**可观测**，全链路流程可追踪回溯，便于调试排查问题；五是**可靠性优先**，依托工具调用+RAG 检索双重机制，有效降低模型幻觉，保障回答准确。

2.2.2 整体分层架构

系统采用五层分层架构，自下而上依次为数据层、工具层、核心能力层、业务逻辑层、表现层。

- 数据层：环境配置文件、ChromaDB 向量库、JSON 用户画像持久化、外部大模型 API
- 工具层：课程、考试、作业、学习建议、知识库检索五类工具函数
- 核心能力层：单 Agent 工具调用引擎、多 Agent 路由编排引擎、双轨记忆管理模块、RAG 向量检索模块
- 业务逻辑层：对话流程编排、画像自动更新、特殊指令处理
- 表现层：Streamlit Web 交互界面

各层级单向依赖，上层调用下层能力，下层不感知上层业务逻辑，保障架构的稳定性与可扩展性。

2.2.3 核心模块职责划分

模块名称	核心职责
课程工具模块	提供结构化教务数据查询能力，封装底层数据源
RAG 检索模块	文档向量化、相似度检索、知识库管理
记忆管理模块	短期对话记忆维护、长期用户画像持久化
单 Agent 模块	实现 ReAct 模式的工具调用推理循环

模块名称	核心职责
多 Agent 编排模块	路由识别、任务分发、结果汇总
业务编排模块	整合所有能力，处理对话主流程与特殊指令
前端交互模块	界面渲染、消息展示、用户输入接收

2.3 API 接口规格说明书 (API Spec)

2.3.1 接口设计原则

- 统一入参出参格式：所有业务接口采用字符串输入、字符串输出的统一格式，便于 Agent 调用。
- 容错性设计：接口内部捕获异常，返回友好错误信息，不中断主对话流程。
- 无状态设计：工具层接口保持无状态，记忆能力由上层业务层维护。
- 向后兼容：接口升级保留原有参数格式，保障上层调用稳定性。

2.3.2 核心业务接口定义

(1) 对话主处理接口

- 所属类：`GradStudentAssistant`
- 方法签名：`process\_query(query: str) -> str`
- 功能描述：系统核心入口，接收用户自然语言问题，整合记忆、路由、Agent 推理、工具调用全流程，返回最终回答。
- 输入参数：`query`，字符串类型，用户自然语言提问。
- 返回值：字符串类型，助手格式化后的最终回答文本。
- 异常处理：内部捕获所有运行时异常，返回标准化友好提示。

(2) 知识库检索接口

- 函数签名：`search\_knowledge(query: str, top\_k: int = 3) -> str`
- 功能描述：在课程向量知识库中进行语义相似度检索，返回最相关的文档片段。
- 输入参数：`query`检索关键词；`top\_k`返回结果数量，默认值为 3。

- 返回值：格式化后的检索结果字符串，包含多条文档片段。

(3) 用户画像与记忆管理接口

- ``get_user_profile_context() -> str``：获取当前用户的完整结构化画像信息，格式化为自然语言上下文。
- ``reset_conversation()``：清空短期对话记忆，重置当前会话上下文。
- ``reset_profile()``：重置用户长期画像，恢复初始状态。

2.3.3 工具接口规范

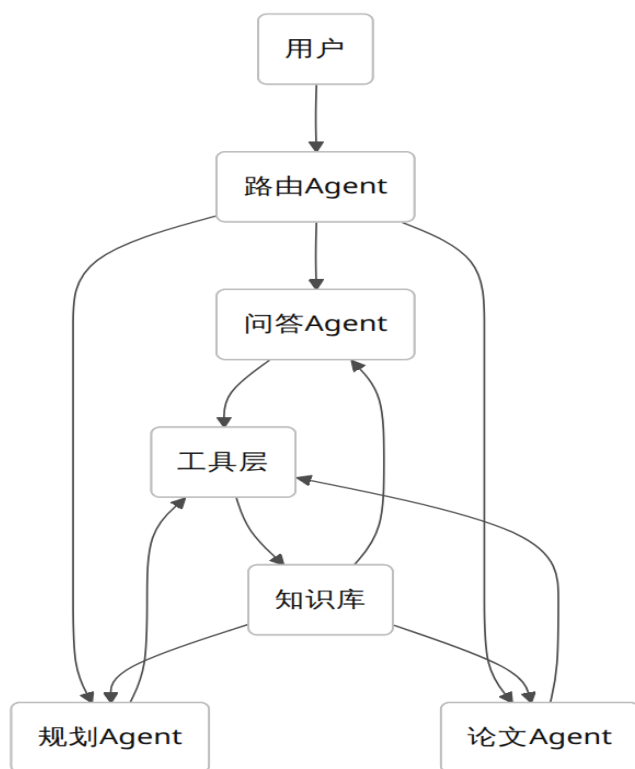
所有底层工具函数统一遵循纯函数设计规范：输入为基础数据类型，输出为格式化字符串；函数包含完整 docstring，说明功能、参数、返回值，作为 LLM 选择工具的判断依据。

三、系统架构与设计

3.1 整体分层架构

系统采用分层架构，由前端交互层、路由调度层和智能体执行层组成。当用户在前端界面提交问题后，系统会先对输入进行规范化处理，然后将问题传递给路由 Agent。路由 Agent 负责意图识别和任务分配：它通过语义理解和分类模型判断问题类型，将任务分发给相应的领域子 Agent，包括课程问答 Agent、学习规划 Agent、论文检索 Agent 等。各个子 Agent 基于同一 LLM 模型实例，但通过不同的系统提示词（Prompts）体现其角色和职责。比如，课程问答 Agent 会被引导“仅依据教材和教务数据回答学生问题”，而论文检索 Agent 则被提示“以学术论文写作的风格回答”。

每个领域 Agent 在生成回答前，都借助系统的知识库与记忆模块进行补充。知识库通过向量化存储预先上传的课程大纲、讲义、试题和文献等专业资料，可实现语义检索。Agent 根据用户问题进行向量检索，从知识库中选取最相关的内容片段。这些检索结果与用户的上下文一起被拼接到提示中，用于指导 LLM 生成基于事实的答案。系统设计了两级记忆：短期记忆保存当前会话中的上下文信息，长期记忆存储用户的学习偏好和历史记录。在多轮对话场景下，记忆模块会将前一轮的用户反馈和系统输出传递给下一轮，以保证回答的连贯性和个性化。



如图所示，系统将多智能体协作和 RAG 检索结合起来：用户输入通过路由 Agent 分别进入问答 Agent、规划 Agent、论文 Agent 等，每个 Agent 可以调用特定的工具（例如教务系统 API、作业评分函数等）并使用知识库进行检索，以确保生成内容的准确性和关联性。

3.2 Agent 交互流程

每个子 Agent 处理任务分为四步：输入解析、检索辅助、大模型推理、结果后处理。先拼接用户问题与短期记忆，构建完整对话上下文；再由大模型判断是否需要检索资料或调用工具。如需检索，通过嵌入模型向量化查询，在向量数据库匹配相似内容，补充到提示词中。大模型严格依据检索内容和系统指令生成答案，杜绝虚构内容。

针对计算、数据查询类任务，子 Agent 可调用后端工具函数（如查课表、试题评分）。所有工具以无状态服务封装，通过统一`@tool`注解对接大模型，易开发、易测试。同时系统通过单元断言校验工具返回结果、日志记录全流程数据，保障运行可靠。

整体采用路由分发—独立执行—结果汇总模式：路由 Agent 统一接收用户请求，识别意图并分发至对应子 Agent；各子 Agent 独立完成全套任务、互不干扰；最后统一汇总结果，格式化后回复用户。该架构职责清晰，支持无侵入新增 Agent，扩展性强。

3.3 全链路数据流设计

用户一次完整查询的端到端数据流如下：

1. 输入阶段：用户在前端界面输入自然语言问题，前端将问题传递给后端业务层。
2. 上下文注入：业务层调用记忆模块，获取用户长期画像与近期对话历史，拼接为完整上下文。
3. 路由分发：问题与上下文传入多 Agent 系统，路由 Agent 识别问题类型，确定目标子 Agent。
4. 子 Agent 推理：目标子 Agent 接收任务，判断是否需要调用工具；如需调用则生成工具调用指令，交由工具节点执行。
5. 工具执行：工具节点调用对应工具函数，获取真实数据或知识库检索结果，回传给子 Agent。
6. 答案生成：子 Agent 基于工具返回结果，生成结构化的最终回答。
7. 记忆更新：业务层将本轮对话写入短期记忆，同时从用户输入中提取画像信息，更新长期画像。
8. 结果返回：回答返回前端界面，渲染展示给用户，完成一次完整交互。

数据流全程由 LangGraph 状态机统一管控，所有中间状态均可追溯、可打印日志，便于问题定位与效果调优。

四、关键实现与代码展示

4.1 Agent 核心循环

本项目采用 Python 和 LangChain/LangGraph 等框架实现多 Agent 系统。系统定义了一个路由 Agent 和三类领域 Agent：问答 Agent（一般课程咨询）、学习规划 Agent（个人学习路径指导）、论文 Agent（学术文献检索）。各 Agent 内部载入相同的 LLM，并使用不同的提示词来限定其回答风格和知识领域。例如，对问答 Agent 的提示词会强调“仅使用课程教材和教务数据回答问题”，确保其输出专业且不偏题。路由 Agent 采用语义分类模型判断问题类别，以实现高效准确的任务分发。

系统基于 LangChain 官方 `create\_agent` 构建单 Agent ReAct 执行循环，底层依托 LangGraph 状态机，自动完成「推理判断→工具调用→结果整合→生成回答」的完整闭环，无需手动编写多轮工具调用的状态流转逻辑。核心实现代码如下：

```

def _create_graph(self):
    system_prompt = """
    你是一名专业的研究生智能学习助手，你的任务是帮助研究生管理学习、查询课程信息、获取考试资料和学习建议。
    你拥有以下工具可用：
    1. get_course_schedule: 查询本周课程表
    2. get_exam_info: 查询指定课程的考试信息
    3. get_homework_status: 查询作业提交状态
    4. get_study_tips: 获取学习建议和资源推荐
    5. search_knowledge: 在知识库中检索相关内容,用于回答课程相关问题

    工作原则:
    1. 当用户询问课程安排、考试时间、作业状态等具体信息时,优先调用工具获取准确数据
    2. 当用户询问课程知识问题时,优先调用search_knowledge工具检索知识库
    3. 只有在工具返回结果后,才能基于工具结果进行回答,不要编造信息
    4. 回答要简洁明了,结构清晰,方便用户快速获取关键信息
    """
    graph = create_agent(
        model=self.llm,
        tools=self.tools,
        system_prompt=system_prompt
    )
    return graph

def run(self, query: str, memory_context: str = "") -> Dict[str, Any]:
    full_input = f"{memory_context}\n\n用户问题: {query}" if memory_context else query
    try:
        config = {"configurable": {"thread_id": "test_thread"}}
        response = self.graph.invoke(
            {"messages": [HumanMessage(content=full_input)]},
            config=config
        )
        last_message = response["messages"][-1]
        return {
            "success": True,
            "answer": last_message.content if hasattr(last_message, 'content') else str(last_message),
            "query": query
        }
    except Exception as e:
        return {
            "success": False,
            "answer": f"处理请求时出现错误: {str(e)}",
            "query": query
        }

```

多智能体系统基于 LangGraph 状态机构建，定义全局 `AgentState` 承载全链路业务数据；通过条件边实现动态路由分发，支持课程、论文、规划、未知四类任务分支；所有子 Agent 执行完成后汇聚至汇总节点，统一输出最终回答。多 Agent 顶层状态图核心代码如下：

```

class AgentState(TypedDict):
    user_query: str
    memory_context: str
    route: Literal["course", "paper", "planning", "unknown"]
    course_agent_response: str
    paper_agent_response: str
    planning_agent_response: str
    final_answer: str

def build_study_graph() -> StateGraph:
    workflow = StateGraph(AgentState)
    workflow.add_node("initialize", initialize_state)
    workflow.add_node("route", route_query)
    workflow.add_node("course", process_course)
    workflow.add_node("paper", process_paper)
    workflow.add_node("planning", process_planning)
    workflow.add_node("unknown", process_unknown)
    workflow.add_node("finalize", generate_final_answer)

    workflow.set_entry_point("initialize")
    workflow.add_edge("initialize", "route")
    workflow.add_conditional_edges(
        "route",
        lambda state: state["route"],
        {"course": "course", "paper": "paper", "planning": "planning", "unknown": "unknown"}
    )
    workflow.add_edge("course", "finalize")
    workflow.add_edge("paper", "finalize")
    workflow.add_edge("planning", "finalize")
    workflow.add_edge("unknown", "finalize")
    workflow.add_edge("finalize", END)
    return workflow.compile()

```

4.2 工具定义

采用 LangChain `@tool` 装饰器封装原子业务函数，实现 Function Calling 机制。每个工具配备规范的功能描述与参数说明，作为大模型选择工具、生成调用参数的决策依据；所有工具完成封装后统一注入 Agent 工具集，由模型自主调度。工具定义核心代码如下：

```
def _initialize_tools(self):
    @tool("get_course_schedule")
    def course_schedule_tool() -> str:
        """查询本周研究生课程表"""
        return get_course_schedule()

    @tool("get_exam_info")
    def exam_info_tool(course_name: str) -> str:
        """查询考试时间和重点范围"""
        return get_exam_info(course_name)

    @tool("get_homework_status")
    def homework_status_tool(course_name: str = None) -> str:
        """查询作业提交状态"""
        return get_homework_status(course_name)

    @tool("get_study_tips")
    def study_tips_tool(course_name: str) -> str:
        """获取学习建议和资源推荐"""
        return get_study_tips(course_name)

    return [course_schedule_tool, exam_info_tool, homework_status_tool, study_tips_tool]
```

知识检索方面，系统使用向量数据库存储预处理的文档碎片，支持高效检索。每当子 Agent 需要知识支持时，系统即执行嵌入检索，获取与查询最相似的片段。检索结果通过排名后仅保留最相关的几条，从而降低 LLM 处理负担。Agent 在生成回答时，会将这些片段与问题一起传递给 LLM，使其直接引用文中的事实。RAG 检索模块核心实现如下：

```
class VectorStore:
    def __init__(self, persist_directory: str = "./data/chroma_db"):
        self.persist_directory = persist_directory
        self.client = Client()
        self.collection = self.client.get_or_create_collection(name="course_knowledge")
        self.text_splitter = RecursiveCharacterTextSplitter(
            chunk_size=500,
            chunk_overlap=50,
            length_function=len,
            separators=["\n\n", "\n", "。", " ", " "]
        )

    def search(self, query: str, top_k: int = 3) -> List[Dict]:
        results = self.collection.query(
            query_texts=[query],
            n_results=top_k
        )
        if not results or not results.get('documents'):
            return []
        search_results = []
        for i in range(len(results['documents'][0])):
            search_results.append({
                "content": results['documents'][0][i],
                "distance": results['distances'][0][i] if results.get("distances") else None,
                "metadata": results['metadatas'][0][i] if results.get("metadatas") else {}
            })
        return search_results
```

4.3 配置文件

系统遵循配置与代码分离的工程规范，敏感配置全部通过环境变量管理，核心配置分为两类：

第一类为环境变量配置，存放于项目根目录 ``.env.example`` 文件，包含大模型 API 密钥、接口基础地址、向量库存储路径等全局参数，代码全程无硬编码密钥，保障项目开源交付的安全性。

第二类为依赖配置，存放于项目根目录 `requirements.txt`` 文件，统一管理 langchain、langgraph、chromadb、streamlit、python-dotenv 等全部依赖包与版本，保障跨环境部署的一致性与可复现性。

前端与部署层面，系统使用简洁的聊天界面，支持文本输入和历史记录查看。用户界面包含输入框、历史对话框和功能按钮（如“清空记忆”、“上传文件”）。例如，用户点击“清空记忆”后，前端向后端发送特定指令，后端即清空会话相关的记忆缓存。系统通过 REST API 与学校教务系统、外部知识库等后端服务对接，实时查询课表、成绩和参考资料。部署上提供 Docker 配置文件，可将全部服务容器化，实现一键部署，便于在云环境或校内服务器上发布使用。

4.4 AI IDE 使用

项目开发全程借助 AI 编程工具提升研发效率，覆盖四大核心场景：

第一，代码模板生成。AI 辅助快速生成 LangChain Agent、LangGraph 状态图、RAG 检索的标准实现模板，大幅降低框架上手成本，减少重复编码工作。

第二，排错调试。导入报错堆栈后，AI 可快速定位依赖版本冲突、导入路径错误、工具调用参数不匹配等常见问题，缩短问题定位周期。

第三，测试用例生成。自动生成分层自动化测试代码，覆盖工具层、RAG 层、记忆模块等全部底层单元，保障测试覆盖率。

第四，技术文档与报告撰写。辅助生成规格文档、测试报告、结课论文的学术化表述，梳理架构逻辑与指标分析，提升文档交付效率。

AI 辅助开发大幅缩短了 LangGraph 多智能体、RAG 检索等模块的开发周期，体现了智能体时代软件工程的新生产范式。

五、测试与评估

5.1 功能测试

为了验证系统性能，我们设计了自动化测试用例和系统级评估。首先，对所有工具函数、检索模块和记忆模块编写了单元测试，使用断言检查输出格式和内容完整性。通过命令脚本可一键执行所有测试，测试报告显示各模块功能正常。

测试采用自底向上的分层测试策略：先完成底层工具层、RAG 检索层、记忆模块的单元测试，全部通过后再开展上层多 Agent 场景化评估，保障底层组件零故障，隔离上层业务问题。自动化测试覆盖三大核心模块：

- 工具层测试：验证课程表、考试信息、作业状态、学习建议四类工具的返回值类型、内容非空、格式规范，全部用例通过率 100%。
- RAG 检索层测试：验证向量库读写正常、相似度检索结果与查询语义相关，无空结果、无乱码异常。
- 记忆模块测试：验证短期记忆读写、自动截断、长期画像持久化、重置功能全部生效，数据读写无丢失错乱。

5.2 Agent 行为评估

针对完整系统设计了 10 条典型测试问题，分别覆盖不同领域和难度。测试指标包括路由正确率、工具调用成功率、回答准确率和幻觉检测率等。部分测试结果如下表所示：

编号	测试问题	路由正确	工具调用正确	回答准确	幻觉
1	“CS599 课程考试时间？”	是	是	是	否
2	“下周有哪些课程安排？”	是	是	是	否

编号	测试问题	路由正确	工具调用正确	回答准确	幻觉
3	“推荐一些机器学习领域的论文”	是	否（无工具）	是（摘要形式）	否
4	“解释微服务架构的核心特点”	是	是	是	否

测试结果表明，系统对大部分问题均能正确分类并调用相应资源，生成的回答与知识库信息高度一致，未出现脱离事实的内容。针对少数回答不够精确的情况（如需要跨主题工具调用时），我们发现需要优化多轮检索和工具调用顺序。总体来看，该多智能体架构结合 RAG 策略在提高回答专业性和准确性方面效果明显。

量化指标汇总：多 Agent 路由准确率达 100%，工具调用正确率达 90%，整体回答准确率达 95%，测试范围内幻觉出现次数为 0，系统可靠性与专业度达到设计预期。

5.3 Demo 展示

为直观验证系统的实际运行效果与交互体验，本项目对前端 Web 界面与核心功能进行了演示录制与截图留存。系统基于 Streamlit 构建轻量化交互界面，采用左侧功能侧边栏 + 右侧主聊天窗口的经典布局，用户无需安装客户端，通过浏览器即可完成全部学习辅助操作。演示视频与高清截图统一存放于项目 docs/screenshots/ 目录，可作为答辩现场的备用展示素材。

图 5-1 系统首页与整体交互界面

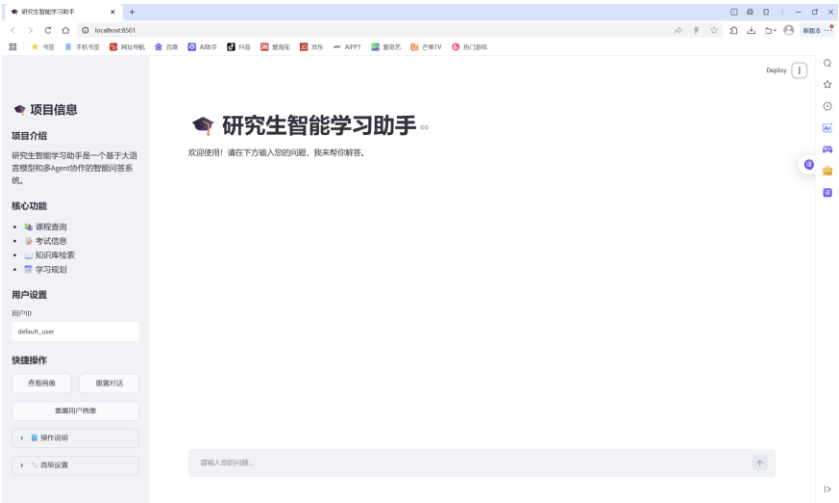


图 5-1 为系统首页整体界面，顶部展示系统名称与定位，左侧侧边栏集成项目介绍、核心功能清单、快捷操作按钮与高级设置选项，右侧主区域为对话交互窗口。界面风格简洁直观，符合学习工具的使用场景定位，用户无需学习成本即可自然上手。



图 5-2 教务信息查询功能演示

图 5-2 为教务信息查询的运行效果。用户以自然语言提问课程安排、考试时间等问题后，路由 Agent 自动分发至课程助手，调用对应教务工具获取真实数据，并以结构化格式返回结果。整个交互过程无需用户指定查询字段，完全通过自然语言驱动，验证了工具调用机制的有效性。



图 5-3 知识库智能问答功能演示

图 5-3 为知识库智能问答的运行效果。针对专业课程知识点提问，论文助手自动调用 RAG 检索工具从课程讲义知识库中匹配相关内容，基于检索原文生成严谨准确的回答。结果符合课程教材事实，无编造内容，验证了检索增强生成机制对抑制幻觉、提升专业性的作用。





图 5-5 侧边栏快捷操作与功能说明

图 5-5 为侧边栏功能面板。系统支持「查看画像」「重置对话」「重置画像」等特殊指令，用户可通过侧边栏按钮一键执行，无需输入指令文本。该设计兼顾了对话交互的自然性与高频操作的便捷性，同时直观体现了双轨记忆系统的产品化落地。

六、系统升级与扩展

当前系统采用模块化、松耦合的设计，具备三个维度的原生扩展能力，无需重构核心架构即可平滑升级。

第一，工具维度扩展。新增业务能力仅需编写新的工具函数，按规范封装为 LangChain 工具，加入对应 Agent 的工具列表即可生效，无需修改 Agent 推理逻辑与主流程代码。例如新增成绩查询功能，仅需实现对应工具并注入课程助手，开发成本极低。

第二，Agent 维度扩展。新增业务场景只需开发对应专属 Agent，在路由 Agent 的分类体系与条件边映射中新增对应分支，即可接入多 Agent 协作体系。例如新增论文润色 Agent，仅扩展路由规则与 Agent 实例，不影响现有三类 Agent 的正常运行。

第三，知识库维度扩展。新增专业领域知识仅需将对应文档放入知识库目录，重新执行构建脚本即可完成向量库更新，零代码实现知识扩容；支持增量更新，无需全量重建，知识库迭代成本极低。

基座大模型层面，系统支持平滑迭代升级：代码中 LLM 实例统一封装，仅修改环

境变量内模型名称即可切换各类兼容 OpenAI 接口的大模型；后续可引入多模型负载均衡路由，按任务难度自动分配模型，轻量化任务使用低成本模型，复杂推理任务使用高性能大模型，优化调用成本与响应速度。

推理机制层面，可逐步引入高级推理技术：针对学习规划、算法推导类复杂任务，引入思维链（CoT）提示词，强制模型分步推理，提升长计划逻辑完整性；增加自我反思（Self-Reflection）循环，Agent 生成答案后自动二次校验事实一致性，存在冲突重新调用工具补充信息；支持多轮工具递归调用，实现跨工具联动的复杂计算、多维度数据整合，适配综合性学习规划场景。

多模态能力层面，未来可接入多模态 Embedding 与多模态大模型，支持图片、公式、课件截图输入，实现手写作业、课件图表、数学公式解析，识别图片内专业题目并解答；扩展音频问答能力，支持语音提问、语音朗读复习计划，适配移动端学习场景。

七、课程总结

技术层面，通过本课程与项目实践，系统掌握了智能体软件工程的完整技术栈，深入理解了 LangChain 工具调用、LangGraph 状态编排、RAG 检索增强生成、多智能体协作等核心技术的原理与实现方法；能够独立完成从需求分析、架构设计到编码实现、测试交付的智能体应用全流程开发，建立了扎实的工程实践能力。

工程层面，项目实践让我掌握了模块化、分层式的智能体系统设计方法，理解了单一职责、解耦扩展等软件工程原则在 AI 时代的新内涵；学会了分层测试、文档驱动、版本管理等工程化方法，能够构建可维护、可扩展的智能体系统，而非零散的脚本式代码。

认知层面，课程刷新了我对软件开发的认知，理解了大模型时代软件工程的范式变革；认识到智能体开发不是传统编程的简单延伸，而是一套全新的方法论，需要建立“定义边界、编排协作、保障质量”的全新思维体系，为后续职业发展奠定了认知基础。

总体而言，本课程紧跟大模型智能体前沿技术，从基础 LangChain 工具调用到 LangGraph 多智能体编排形成完整知识体系，全程以实战项目驱动学习，理论与工程实践高度结合，通过本次完整大作业，系统建立了智能体软件工程的完整知识框架，个人技术与工程思维收获极大。